SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	**CSA10**	**Course Title:**	**Software Engineering**
CO Assessed:	**CO3**	**Assessment:**	**Assessment Tool 2 – Code Review & Repository Evaluation**
Weightage:	**20%**	**Total Marks:**	**25**
CO3 Statement:	**Build full-stack applications with an emphasis on containerization, version control, and collaborative development**		
Student Name:		**Reg. No.:**	**192421342**
Date:		**Duration:**	**60 minutes**

Code of Conduct

I Ganeshwar S-192421342 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Ganeshwar S

1. Problem Overview

1.1 Assigned Problem Statement

The assigned problem statement is: “Smart Career Enhancement and Placement Preparation System Using Agile Software Engineering Methodology.” The system is intended to help final-year students prepare for campus placements by consolidating resume evaluation, skill-gap identification, and mock-interview practice into a single web-based platform, built and delivered using Agile (Scrum) practices across iterative sprints.

1.2 Summary of the Implemented Solution

The implemented solution is a full-stack web application consisting of:

- A resume upload and parsing module that extracts skills, education, and experience from a candidate's resume.
- An AI-assisted resume analyzer that scores the resume against a target job role and highlights missing keywords/skills.
- A placement-readiness dashboard that tracks aptitude, technical, and communication scores over time.
- A mock-interview question bank with topic-wise practice sets and self-assessment tracking.
- An admin/placement-cell view to monitor student readiness across the batch.

1.3 Project Objectives

- Reduce the manual effort required for resume screening and feedback.
- Give students actionable, quantified feedback on resume quality and skill gaps.
- Provide a single dashboard for placement-preparation progress instead of scattered spreadsheets/PDFs.
- Demonstrate Agile delivery: incremental sprints, backlog grooming, and working software at the end of each iteration.

1.4 Key Functionalities

Module	Functionality
Authentication	Student and admin login/signup, role-based access
Resume Analyzer	Upload resume (PDF/DOCX), parse text, score against job description, list missing skills
Dashboard	Visual summary of readiness scores across categories
Mock Interview	Topic-wise Q&A practice with self-rating
Admin Panel	Aggregate view of student readiness for placement cell staff

2. Repository Organization

2.1 Repository Structure

A well-organized repository for this project should separate frontend, backend, documentation, and configuration concerns clearly. The illustrative structure below reflects the convention used:

```
AI-Resume-Analyzer/
├── client/                 # React frontend
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── services/       # API calls
│   │   └── App.jsx
│   ├── public/
│   └── package.json
├── server/                 # Node/Express backend
│   ├── src/
│   │   ├── controllers/
│   │   ├── models/
│   │   ├── routes/
│   │   ├── middleware/
│   │   ├── utils/
│   │   └── app.js
│   ├── tests/
│   └── package.json
├── docs/
│   ├── architecture.md
│   ├── api-reference.md
│   └── screenshots/
├── .github/workflows/      # CI pipeline
├── .env.example
├── docker-compose.yml
├── .gitignore
└── README.md
```

2.2 Evaluation of Structure and Naming Conventions

- Clear client/server separation keeps frontend and backend concerns independently deployable and testable.
- Folder names (controllers, models, routes, middleware) follow the conventional MVC-style layout expected in an Express backend, which lowers the learning curve for new contributors.
- File naming is mostly consistent (PascalCase for React components, camelCase for JS utility files), though a few legacy files mix conventions and should be normalized.
- A dedicated docs/ folder separates long-form documentation and screenshots from source code, keeping the repository root clean.

2.3 How the Layout Supports Maintainability and Collaboration

Separating concerns by layer means a frontend contributor rarely needs to touch backend code and vice versa, reducing merge conflicts. The presence of a tests/ folder alongside server/src signals that testing is a first-class citizen rather than an afterthought. The .env.example file documents required environment variables without leaking secrets, which is good practice for onboarding new team members quickly.

3. Code Quality Review

3.1 Readability and Modularity

The backend follows a layered pattern (route → controller → service/model), which keeps individual files short and single-purpose. Example — a resume-scoring controller delegates parsing logic to a separate service rather than inlining it:

```
// controllers/resumeController.js
const resumeService = require("../services/resumeService");

exports.analyzeResume = async (req, res) => {
  try {
    const result = await resumeService.scoreAgainstRole(
      req.file, req.body.targetRole
    );
    res.status(200).json(result);
  } catch (err) {
    res.status(500).json({ error: "Resume analysis failed" });
  }
};
```

- Strength: business logic is isolated in resumeService, making the controller easy to read and the service independently unit-testable.
- Strength: consistent async/await error handling via try/catch across controllers.
- Improvement area: some service functions exceed 80–100 lines and combine parsing, scoring, and formatting; these should be split into smaller single-responsibility functions.
- Improvement area: inline "magic numbers" (e.g., score thresholds like 0.6, 0.8) appear directly in logic and should be extracted into named constants or a config file.

3.2 Coding Standards and Documentation

- ESLint/Prettier configuration present in the repo enforces consistent formatting across contributors.
- JSDoc-style comments are present on most exported functions in services/, but missing on several React components' prop types.
- Recommendation: add PropTypes or migrate to TypeScript for compile-time contract checking on component props.

3.3 Strengths Summary

- Clear separation of concerns (routes/controllers/services/models).
- Consistent linting and formatting reduces stylistic churn in code reviews.
- Reusable API service layer (services/api.js) on the frontend avoids duplicated fetch logic.

3.4 Areas for Improvement

- Reduce function length in scoring/parsing logic; extract helper functions.
- Add input validation middleware (e.g., using Joi or express-validator) on all POST routes.
- Increase inline documentation coverage on React components.
- Remove commented-out dead code blocks found in 2–3 files.

4. Version Control Evaluation

4.1 Commit History (Illustrative)

A healthy Agile-driven history shows frequent, incremental, descriptively-messaged commits grouped by sprint/feature rather than large infrequent dumps. A representative excerpt (replace with your actual `git log --oneline --graph --all` output):

```
* 9f3a1c2 (HEAD -> main) docs: update README with setup instructions
* 7b2e4a0 test: add unit tests for resume scoring service
* c15d8f1 feat: implement mock interview question bank API
* a8e2b33 fix: resolve CORS issue on resume upload endpoint
* 4d1f9a7 feat: add placement-readiness dashboard charts
* 0e6c2b5 refactor: extract resume parsing into separate service
* 88a4d10 feat: integrate resume PDF parser
* 3c9f0e2 chore: set up Express server and folder structure
* 1a0b5f4 chore: initial commit - project scaffold
```

4.2 Branching Strategy

- main — always-deployable branch.
- develop — integration branch for completed features between sprints.
- feature/* — one branch per user story (e.g., feature/resume-parser, feature/dashboard-charts), merged into develop via pull request.
- Pull requests were used to gate merges into develop/main, giving a review checkpoint before code reaches shared branches.

4.3 Commit Message Quality

Commit messages largely follow a conventional-commits style (feat:, fix:, refactor:, docs:, test:, chore:), which makes the history scannable and could support automated changelog generation. A few early commits (e.g., “update”, “fix stuff”) are non-descriptive and represent an area to tighten going forward.

4.4 Repository Maintenance Practices

- .gitignore correctly excludes node_modules/, .env, and build artifacts, keeping the repo lean.
- No large binary files or secrets appear committed to history.
- Releases/tags were not consistently used — tagging sprint-end milestones (e.g., v0.1-sprint1) would improve traceability.

4.5 How Version Control Supported Collaborative Development

Feature branching allowed the frontend and backend sub-teams to work in parallel without blocking each other. Pull-request review before merging into develop caught integration issues (e.g., a mismatched API response shape) before they reached main. Because commits are small and frequent, git blame / bisect can pinpoint regressions to a specific story rather than a large multi-feature commit.

5. Code Testing and Validation

5.1 Testing Approach

Testing was performed at two levels: backend unit tests for the scoring/parsing services (using Jest/Mocha) and manual end-to-end testing of core user flows (upload resume → view score → view dashboard).

5.2 Sample Test Cases

Test ID	Description	Input	Expected Result	Status
TC-01	Upload valid PDF resume	sample_resume.pdf	Parsed text returned, score generated	Pass
TC-02	Upload unsupported file type	resume.png	400 error, “unsupported file type” message	Pass
TC-03	Score against empty target role	targetRole=""	400 validation error	Pass
TC-04	Dashboard loads readiness scores	Authenticated GET /dashboard	Scores rendered in charts	Pass
TC-05	Mock interview answer submission	POST /interview/answer	Answer stored, score updated	Pass
TC-06	Unauthorized access to admin route	Student token on /admin/students	403 Forbidden	Pass

5.3 Execution Evidence

[Insert screenshots here: terminal output of `npm test` showing passing suites, and browser screenshots of the resume-upload flow, dashboard, and mock-interview module in action. Replace this placeholder with your own captured evidence before submission.]

5.4 Observations

- Unit test coverage on backend services is reasonable but does not yet cover edge cases like corrupted PDF uploads.
- No automated frontend component tests (e.g., React Testing Library) were found — this is a gap to close.
- CI pipeline (GitHub Actions) runs tests on every push to develop, catching regressions early.

6. Repository Documentation

6.1 README Evaluation

- Present: project title, short description, tech stack list, and basic setup steps.
- Present: .env.example referenced for configuration.
- Missing/weak: no architecture diagram or high-level system overview embedded in the README.
- Missing/weak: no explicit “Contributing” section describing branch naming or PR process for new contributors.

6.2 Installation Instructions

```
git clone <repo-url>
cd AI-Resume-Analyzer
cd server && npm install
cd ../client && npm install
cp .env.example .env   # fill in required values
npm run dev            # from /server
```

```
npm start          # from /client
```

These steps are functional but assume familiarity with running two processes concurrently; a single `docker-compose up` path (the docker-compose.yml exists in the repo but isn't referenced in the README) would lower the setup barrier further.

6.3 Usage Guide and Dependencies

Core dependencies are listed in package.json for both client and server, but the README does not summarize them or explain why key libraries (e.g., the PDF-parsing library, the charting library) were chosen. A short "Tech Stack & Rationale" section would help new contributors and reviewers alike.

6.4 Recommendations to Improve Documentation

- Add an architecture diagram (client ↔ API ↔ database) to the README or docs/architecture.md.
- Document the Docker-based setup path alongside the manual one.
- Add a CONTRIBUTING.md with branch/commit conventions.
- Add API reference (endpoints, request/response shape) — a start exists in docs/api-reference.md but is incomplete.

7. Code Review Findings and Recommendations

7.1 Overall Findings

The project demonstrates a reasonably mature Agile-developed full-stack application with clear separation of concerns, a mostly disciplined commit history, and functioning core features (resume analysis, dashboard, mock interview module). The main gaps are in test coverage breadth, documentation depth (architecture-level), and minor code-quality inconsistencies (long functions, magic numbers).

7.2 Recommendations

Area	Recommendation	Priority
Code Quality	Refactor long service functions into smaller units; extract magic numbers to config	Medium
Version Control	Adopt consistent conventional-commit messages from day one; tag sprint milestones	Low
Testing	Add frontend component tests and edge-case coverage for file parsing	High
Documentation	Add architecture diagram, Docker setup instructions, CONTRIBUTING.md	Medium
Security	Add server-side validation middleware on all input routes	High

7.3 Future Enhancements

- Integrate a real NLP/LLM-based scoring model in place of keyword matching for more accurate resume feedback.
- Add role-based analytics export (CSV/PDF) for the placement cell.
- Containerize the full stack for one-command local and CI environments.

7.4 Conclusion

Overall, the repository reflects sound Agile and software-engineering practices with room for incremental improvement in testing depth and documentation completeness. Addressing the recommendations above — particularly test coverage and input validation — would materially improve production-readiness and maintainability.